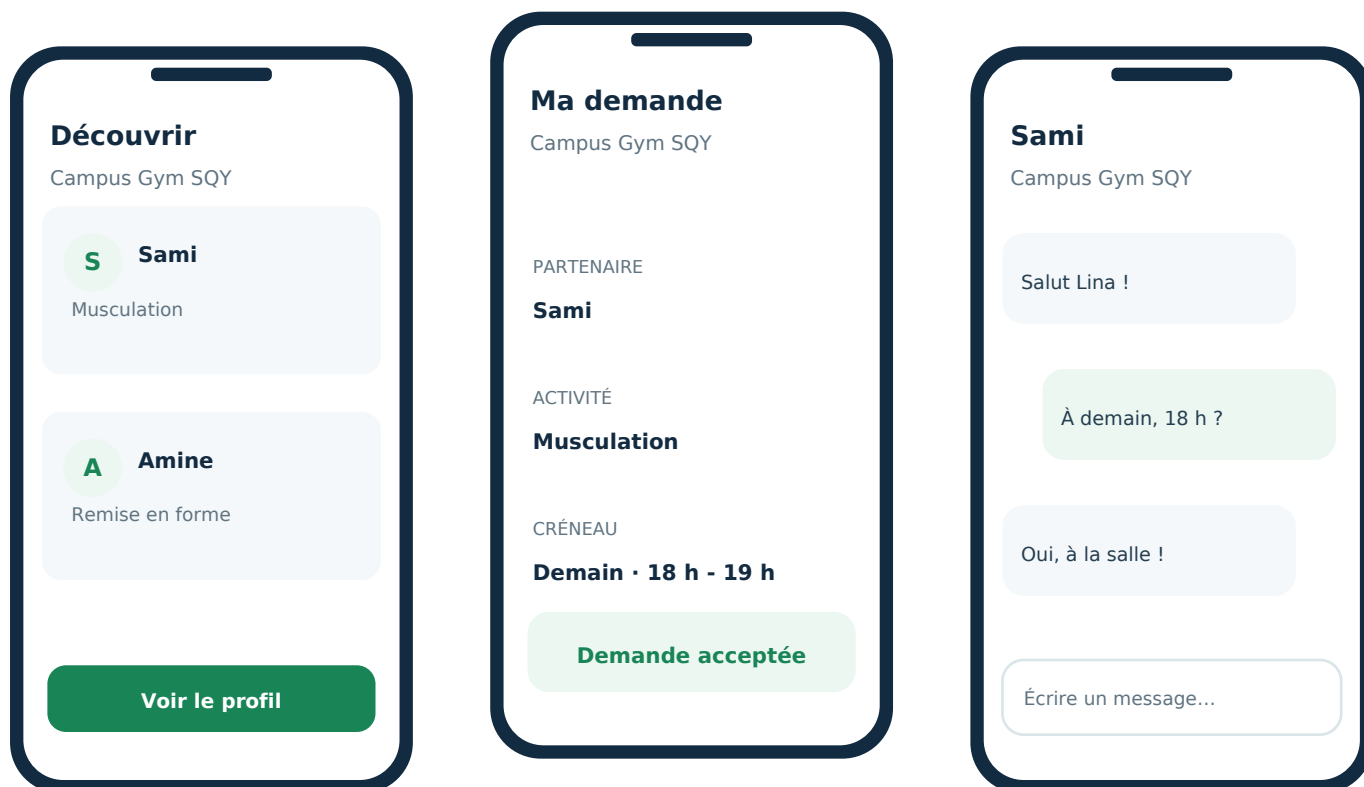


GymMate

Trouve ton partenaire de sport.

Concevez une application qui met en relation les membres d'une même salle, organise leurs séances et ouvre un chat après acceptation.



Écrans indicatifs : à concevoir et coder par les étudiants.

REACT NATIVE · EXPO · TYPESCRIPT · FIREBASE

32 pages / 10 jalons / consignes, tests et livrables

DIGINOVA | Hanane SADEG

Projet fil rouge à développer tout au long de la formation

Pitch du projet

Votre mission : concevoir une application qui aide les membres d'une même salle de sport à organiser des entraînements ensemble.

01 Le besoin à résoudre

Une personne souhaite s'entraîner régulièrement, mais ne connaît pas les autres membres de sa salle. Elle cherche un partenaire avec un niveau, des objectifs ou des disponibilités compatibles.

GymMate met les utilisateurs en relation à partir d'une demande d'entraînement concrète : un sport, une date et un créneau. Le destinataire choisit d'accepter ou de refuser.

02 Périmètre de la version à rendre

Obligatoire	Après validation du socle
Compte, profil sportif et choix d'une salle.	Photo de profil avec Storage.
Membres de la même salle et fiche détaillée.	Géolocalisation et recherche par distance.
Demandes, acceptation, refus et relations.	Notifications et rappels de séance.
Séances prévues, chat en temps réel et règles d'accès.	Calendrier, filtres avancés et blocage.

03 Cadre de réalisation

Développez avec **React Native, Expo, TypeScript et Firebase**. Utilisez Firebase Authentication pour les comptes et Cloud Firestore pour les données. Une liste de salles fictives est fournie page 20.

Conservez un seul dépôt Git pendant la formation. Les jalons se valident en cours ; leur durée dépend de la progression de la classe. Chaque fonctionnalité doit pouvoir être expliquée et démontrée.

RÉSULTAT ATTENDU

Deux utilisateurs de la même salle peuvent se découvrir et préparer une séance commune.

Objectifs pédagogiques

Construisez d'abord une interface locale, puis connectez progressivement ses fonctionnalités à Firebase. Les jalons ci-dessous donnent l'ordre de réalisation.

01 Progression sur l'ensemble de la formation

Jalon	Notions à pratiquer	Preuve / pages utiles
J1 · Démarrer	Expo, JSX, View, Text, StyleSheet.	Accueil sur mobile · p. 14 et 4
J2 · Interagir	Composants, props, state, formulaires, listes.	Profil local + cartes · p. 4, 19, 21
J3 · Naviguer	Routes, onglets, paramètres, retour.	Parcours cliquable · p. 5-6
J4 · Connecter	Configuration, services, contexte, effets.	Firebase prêt · p. 8-16
J5 · Identifier	Auth, validation, profil persistant.	Compte A utilisable · p. 17-19
J6 · Découvrir	Lecture, requête filtrée, états d'écran.	A voit B, pas C · p. 20-21
J7 · Demander	Écriture, modèles, statuts métier.	Demande reçue par B · p. 22-23
J8 · Relier	Transaction, identifiants, cohérence.	Relation + séance · p. 24-26
J9 · Échanger	onSnapshot, nettoyage, messages.	Chat A/B en direct · p. 27-28
J10 · Livrer	Règles, tests, README, démonstration.	Rendu vérifiable · p. 29-32

02 Méthode à suivre à chaque jalon

Comprendre → coder → tester → expliquer → faire un commit. Notez dans docs/suivi.md ce qui fonctionne, le test réalisé et la difficulté rencontrée. Les règles d'accès se construisent avec les fonctionnalités et sont vérifiées au jalon final.

RÉSULTAT ATTENDU

À chaque étape, une nouvelle fonctionnalité fonctionne et une preuve figure dans le suivi.

Résultat final attendu

À la fin du projet, l'application réalise le parcours complet avec des données persistantes et deux comptes distincts.

01 Écrans et critères visibles

Zone	Ce que l'utilisateur peut faire
Bienvenue / compte	S'inscrire, se connecter, voir une erreur compréhensible.
Profil / salle	Enregistrer son profil sportif et sélectionner une salle.
Découvrir	Lire les cartes des autres membres de sa salle et leur profil.
Demandes	Proposer un créneau, suivre une demande, accepter ou refuser.
Relations / séances	Retrouver ses partenaires et les entraînements planifiés.
Messages	Discuter avec un partenaire après une première acceptation.

02 Premier travail React Native, sans Firebase

1. Dessinez trois écrans : membres, détail d'un sportif, formulaire de demande. Repérez les éléments réutilisables.
2. Créez MemberCard, PrimaryButton et EmptyState. Faites circuler les données par les props.
3. Affichez trois sportifs fictifs avec FlatList. Utilisez useState pour le formulaire et un état de chargement simulé.
4. Faites fonctionner les boutons et la navigation. Les tableaux locaux seront remplacés par les services Firebase.

03 Qualité minimale de chaque écran

Prévoyez les états **chargement**, **vide**, **succès** et **erreur**. Les textes restent lisibles, le clavier ne masque pas la validation, les boutons ont un libellé explicite et un double appui ne duplique pas l'action.

Repère technique : [React Native : listes](#)

RÉSULTAT ATTENDU

L'interface locale est utilisable avant la connexion aux données distantes.

Parcours utilisateur

Le consentement du destinataire est le point central du projet. Une simple visite de profil ne crée aucune relation.

DEMANDE EN ATTENTE

A propose une séance à B : statut pending

B refuse

rejected : aucun accès créé

B accepte

accepted : validation atomique

Relation + séance + chat

Accès réservé aux deux membres

01 Navigation à construire

Situation	Destination attendue
Utilisateur déconnecté	Bienvenue, inscription ou connexion.
Compte connecté, profil incomplet	Compléter le profil, puis choisir une salle.
Compte prêt	Onglets Découvrir, Demandes, Séances, Messages, Profil.
Carte membre sélectionnée	Détail du sportif, puis proposition d'entraînement.
Demande acceptée	Relation visible, séance créée, conversation accessible.

02 À coder et à vérifier

Transmettez les **identifiants** dans les routes : `userId`, `requestId` ou `chatId`. L'écran relit les données autorisées. Vérifiez le bouton Retour, une route inexistante et une ouverture directe du chat avant acceptation.

Une relation déjà créée reste disponible si une nouvelle demande est refusée ou si une séance est annulée. La suppression d'une relation est un bonus.

RÉSULTAT ATTENDU

Le parcours A → demande → décision de B → relation → chat est explicite et testable.

Architecture React Native + Firebase

Séparez l’affichage, la logique de l’application et l’accès aux services. Firebase prend en charge le backend du projet.

APPLICATION MOBILE

Écrans React Native + navigation Expo Router + état React



SERVICES TYPESCRIPT

authService · profileService · requestService · chatService



Authentication

Identité et session

Cloud Firestore

Données + écoute en temps réel

01 Organisation conseillée

Emplacement	Responsabilité
src/app/	Routes et écrans Expo Router ; _layout.tsx pour les layouts.
src/components/	Cartes, champs, boutons et états communs.
src/contexts/	AuthContext et accès à l'utilisateur connecté.
src/services/	Initialisation Firebase et fonctions de lecture / écriture.
src/types/ · src/utils/	Types métier, validation et formatage des dates.

02 À réaliser

Utilisez **Expo Router** et conservez le dossier de routes du template (app/ ou src/app/). Placez le fournisseur d'authentification à la racine.

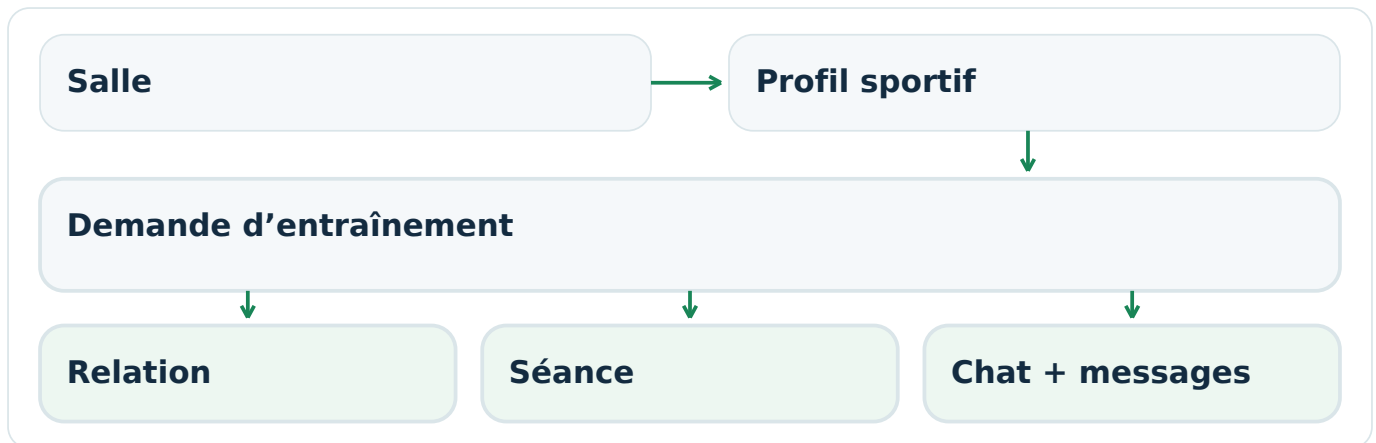
L'écran appelle par exemple `sendTrainingRequest(input)`. Le service prépare l'écriture ; le composant affiche le succès ou l'erreur.

RÉSULTAT ATTENDU

Les accès Firebase sont regroupés dans les services et les composants restent réutilisables.

Modèle de données global

Firestore stocke des documents dans des collections. Les identifiants relient les documents ; les règles en contrôlent l'accès.



01 Collections du socle

Chemin	Rôle et identifiants
users/{uid}	Profil sportif partageable ; uid = identifiant Authentication.
gyms/{gymId}	Catalogue de salles préparé depuis la console.
trainingRequests/{requestId}	Proposition, deux participants, date, statut.
relations/{pairId}	Lien unique entre deux utilisateurs ayant accepté une demande.
trainingSessions/{requestId}	Séance issue de cette demande acceptée.
chats/{pairId}/messages/{id}	Un chat par relation ; un document par message.

02 Choix de conception à respecter

Le mot de passe et l'email restent dans Authentication. N'ajoutez aucune donnée privée aux profils lisibles par les autres membres. Si vous ajoutez plus tard des informations privées, placez-les dans des documents réservés à leur propriétaire.

Utilisez des Timestamp pour les instants et serverTimestamp() pour les dates de création. Les noms affichés proviennent des profils ; les autorisations reposent sur les UID.

Repère technique : [Firestore : accès aux champs](#)

RÉSULTAT ATTENDU

Chaque collection a un rôle clair et les profils partagés ne contiennent pas de coordonnées privées.

Création du projet Firebase

Cette préparation intervient au jalon J4, une fois les premiers écrans locaux et leur navigation réalisés.

01 Préparer votre environnement de projet

1. Ouvrez la console Firebase avec le compte utilisé pour la formation.
2. Créez un projet dédié, avec un nom reconnaissable, par exemple gymmate-groupe-03.
3. Notez le **Project ID** exact dans vos notes de configuration. Il identifie le backend partagé par les comptes de test.
4. Terminez l'assistant. Google Analytics n'est pas nécessaire pour les fonctionnalités du socle.
5. Repérez les rubriques Paramètres du projet, Authentication et Firestore Database. Leurs intitulés peuvent varier selon la langue de la console.

02 Organisation pour les essais

Élément	Choix de travail
Un rendu	Un dépôt Git et un projet Firebase clairement identifiés.
Démonstration A/B	Deux comptes Authentication dans le même projet Firebase.
Jeu de données	Profils et salles fictifs ; créneaux situés dans le futur.
Traçabilité	Versions des dépendances et configuration décrites dans le README.

03 Mission et point de contrôle

Dans docs/suivi.md, expliquez la différence entre le projet Expo installé sur votre poste et le projet Firebase accessible à distance.

À vérifier : vous accédez au bon projet, son nom est identifiable et vos futurs appareils de test utiliseront le même projectId.

Repère technique : [Console Firebase](#)

RÉSULTAT ATTENDU

Le projet Firebase est créé et vous savez où configurer les comptes et la base.

Ajout d'une application Web Firebase

Pour utiliser le SDK JavaScript Firebase dans Expo, enregistrez une application Web à l'intérieur du projet Firebase.

01 Enregistrer l'application

1. Dans la vue d'ensemble du projet, choisissez Ajouter une application, puis l'icône Web `</>`.
2. Saisissez un nom, par exemple `gymmate-mobile-js`.
3. Laissez Firebase Hosting de côté pour ce projet mobile.
4. Validez l'enregistrement et affichez la configuration du SDK JavaScript.
5. Repérez l'objet `firebaseConfig`. Vous le retrouverez dans Paramètres du projet, puis Vos applications.

02 Comprendre le choix technique

L'application que vous codez reste une application **React Native**. L'enregistrement Web fournit les paramètres utilisés par le package JavaScript `firebase` dans Expo.

Le socle utilise ce SDK pour Authentication et Firestore. Conservez ce choix dans tous vos fichiers : les exemples du dossier reposent sur les imports modulaires `firebase/app`, `firebase/auth` et `firebase/firestore`.

03 À ne pas confondre

Élément	Signification dans GymMate
Projet Firebase	L'espace contenant les services et les données du projet.
Application Web enregistrée	Une configuration cliente rattachée à cet espace.
Application Expo	Le code mobile développé et exécuté par les étudiants.
Compte utilisateur	Une identité créée ensuite dans Authentication.

04 Vérification

Expliquez en une phrase pourquoi vous avez choisi l'icône Web alors que le résultat attendu est mobile. Identifiez le `projectId` dans la configuration et comparez-le à celui de la page précédente.

Repère technique : [Expo : utiliser Firebase](#)

RÉSULTAT ATTENDU

Une application cliente est enregistrée et son objet `firebaseConfig` est accessible.

Récupération config Firebase

Centralisez les paramètres du projet pour pouvoir installer GymMate sur un autre poste sans modifier les écrans.

01 Reporter les valeurs de la console

Champ Firebase	Variable dans .env.local
apiKey	EXPO_PUBLIC_FIREBASE_API_KEY
authDomain	EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN
projectId	EXPO_PUBLIC_FIREBASE_PROJECT_ID
appId	EXPO_PUBLIC_FIREBASE_APP_ID

02 Créer src/services/firebaseConfig.ts

```
export const firebaseConfig = {  
  apiKey: process.env.EXPO_PUBLIC_FIREBASE_API_KEY,  
  authDomain: process.env.EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN,  
  projectId: process.env.EXPO_PUBLIC_FIREBASE_PROJECT_ID,  
  appId: process.env.EXPO_PUBLIC_FIREBASE_APP_ID,  
};
```

Créez `.env.local` à la racine d'Expo avec les valeurs de la console. Si le projet Expo n'existe pas encore, faites cette étape après la page 14.

03 À réaliser dans le dépôt

1. Ajoutez `.env.local` aux fichiers ignorés par Git. Créez un `.env.example` avec les mêmes noms et des valeurs vides.
2. Vérifiez que les quatre valeurs sont définies avant d'initialiser Firebase. Affichez une erreur de configuration explicite si l'une manque.
3. Après un changement de configuration, relancez Expo et rechargez complètement l'application.

04 Ce qui protège réellement les données

Les variables `EXPO_PUBLIC_` sont intégrées au code client et sont donc lisibles. La configuration cliente Firebase n'est pas une clé d'administration. **Les accès sont protégés par Authentication et les Security Rules.** Aucun fichier de compte de service ni clé privée ne doit être placé dans l'application.

Repère technique : [Expo : variables d'environnement](#)

RÉSULTAT ATTENDU

La configuration est centralisée, documentée et indépendante du code des écrans.

Activation d'Authentication

Authentication gère les identités. Le profil sportif, lui, sera enregistré dans Firestore.

01 Préparer le service

1. Ouvrez Authentication dans la console du projet.
2. Lancez la configuration du service si elle n'a pas encore été effectuée.
3. Repérez la zone des fournisseurs de connexion et l'onglet des utilisateurs.
4. Passez à la page 12 pour activer Email/Password avant les essais d'inscription.

02 Répartir les responsabilités

Authentication	Cloud Firestore
Identifiant unique uid.	Document users/{uid}.
Email et mécanisme d'authentification.	Nom affiché, niveau et objectif sportif.
Vérification du mot de passe.	Sports, disponibilités, bio et salle.
État de la session connectée.	Demandes, relations, séances et messages.

03 À préparer dans le code

Définissez les opérations attendues du service d'authentification : `register(email, password)`, `login(email, password)` et `logout()`.

Créez le type de profil séparément du type User fourni par Firebase. Un compte peut exister alors que son profil sportif n'a pas encore été complété.

04 Question de compréhension

Que doit faire l'application si un utilisateur est connecté, mais que `users/{uid}` n'existe pas encore ? Prévoyez une étape de création ou de reprise du profil, avec un message compréhensible.

À vérifier : vous savez retrouver l'UID d'un compte et expliquer comment il deviendra l'identifiant de son document de profil.

Repère technique : [Firebase : authentification par mot de passe](#)

RÉSULTAT ATTENDU

Vous distinguez une identité connectée d'un profil métier et savez les relier par l'UID.

Activation d'Email/Password

La version obligatoire utilise une connexion par email et mot de passe pour permettre des essais reproductibles.

01 Activer le fournisseur

1. Dans Authentication, ouvrez les méthodes de connexion.
2. Sélectionnez Email/Password ou Adresse e-mail/Mot de passe.
3. Activez la connexion classique par mot de passe et enregistrez.
4. Relevez la politique de mot de passe configurée. Votre formulaire doit informer l'utilisateur des critères attendus.
5. Vérifiez que ce fournisseur apparaît comme activé avant de développer l'inscription.

02 Préparer les cas de test

Essai	Résultat attendu
Email valide, mot de passe conforme	Compte créé au jalon d'inscription.
Email mal formé	Message explicite et correction possible.
Mot de passe non conforme	Critères attendus indiqués à l'utilisateur.
Adresse déjà utilisée	Échec pris en charge sans bloquer le formulaire.
Identifiants incorrects à la connexion	Message générique : connexion impossible avec ces identifiants.

03 Consignes pour les formulaires

Utilisez `secureTextEntry` pour le mot de passe et désactivez la majuscule automatique pour l'email. Vous pouvez nettoyer les espaces de l'email ; **ne modifiez pas silencieusement le mot de passe.**

Désactivez le bouton pendant l'envoi. Gérez `try / catch / finally` pour réactiver l'écran après un succès comme après une erreur.

04 Point de contrôle

Votre interface montre un état d'attente et un message utile. Les détails techniques d'une erreur restent dans les outils de diagnostic, sans afficher de mot de passe.

RÉSULTAT ATTENDU

Email/Password est activé et les comportements attendus des formulaires sont définis.

Création de Firestore Database

Préparez la base qui recevra les profils et toutes les données du parcours sportif.

01 Créer la base et son catalogue

1. Ouvrez Firestore Database, puis Créer une base. Choisissez la base par défaut et une configuration compatible avec les collections et documents du SDK Firestore.
2. Choisissez une localisation proposée adaptée au projet de formation ; notez ce choix dans le README.
3. Démarrez avec des règles fermées. Les droits seront ouverts collection par collection pendant les jalons.
4. Dans l'onglet Données, créez les trois documents gyms décrits page 20. Leur préparation depuis la console ne nécessite pas d'autoriser l'écriture depuis l'application.

02 Premier accès autorisé : lire les salles

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /gyms/{gymId} {
      allow read: if request.auth != null;
      allow write: if false;
    }
  }
}
```

Enregistrez cette base dans firestore.rules, puis publiez-la dans la console. Les chemins sans autorisation restent refusés. Complétez ensuite les règles des profils et des demandes selon la page 29.

03 Comprendre un refus d'accès

permission-denied peut être normal : utilisateur déconnecté, règle non ajoutée ou requête incompatible avec les droits. Vérifiez l'UID, le chemin demandé et la règle concernée.

Un profil peut être lisible par son propriétaire et les membres autorisés de sa salle. Une demande doit être lisible uniquement par son expéditeur et son destinataire. Ces contrôles s'ajoutent avant les tests de chaque fonctionnalité.

Repère technique : [Firestore : démarrage](#)

RÉSULTAT ATTENDU

La base et le catalogue existent ; les données ne sont pas ouvertes à tous.

Installation d'Expo + Firebase

Commencez ici au jalon J1. Ajoutez Firebase et AsyncStorage au jalon J4, quand les premières interfaces locales fonctionnent.

01 Commandes à saisir dans un terminal

```
node --version
npm --version
npx create-expo-app@latest gymmate --template default
cd gymmate
npx expo start
```

Utilisez une version de Node.js compatible avec le SDK Expo créé. Le template default fournit TypeScript et Expo Router. Ouvrez l'application avec un environnement mobile compatible avec ce SDK.

02 Votre premier objectif mobile

1. Repérez la route d'accueil créée par le template et remplacez son contenu par le nom GymMate, un texte de bienvenue et un bouton Découvrir.
2. Construisez MemberCard avec View, Text, Pressable et StyleSheet. Ajoutez ensuite la liste locale demandée page 4.
3. Créez une route de détail et vérifiez l'aller-retour. Faites un premier commit lorsque ces écrans fonctionnent.

03 Au jalon J4 : ajouter les dépendances Firebase

```
npx expo install firebase
npx expo install @react-native-async-storage/async-storage
npx expo-doctor
```

Conservez package-lock.json dans Git et notez les versions installées. Si Expo Go ne correspond pas au SDK du projet, utilisez l'environnement compatible indiqué par le cours.

04 Si l'application ne démarre pas

Lisez la première erreur du terminal. Vérifiez le dossier courant, l'installation des dépendances et la connexion de l'appareil. Après une correction, un redémarrage avec `npx expo start --clear` peut aider.

Repère technique : [Expo : créer un projet](#)

RÉSULTAT ATTENDU

GymMate démarre sur un appareil ou un émulateur et affiche vos premiers composants.

Configuration de firebase.ts

Initialisez les services à un seul endroit. L'exemple suivant cible l'application mobile Android / iOS et conserve la session avec AsyncStorage.

01 Créer src/services/firebase.ts

```
import { getApp, getApps, initializeApp } from 'firebase/app';
import {
  getAuth, initializeAuth, getReactNativePersistence,
} from 'firebase/auth';
import { getFirestore } from 'firebase/firestore';
import AsyncStorage from '@react-native-async-storage/async-storage';
import { firebaseConfig } from '../firebaseConfig';

const app = getApps().length
  ? getApp() : initializeApp(firebaseConfig);

const mobileAuth = () => {
  try {
    return initializeAuth(app, {
      persistence: getReactNativePersistence(AsyncStorage),
    });
  } catch (error) {
    if ((error as { code?: string }).code
      !== 'auth/already-initialized') throw error;
    return getAuth(app);
  }
};

export const auth = mobileAuth();
export const db = getFirestore(app);
```

02 À intégrer et à vérifier

1. Importez toujours auth et db depuis ce fichier dans vos services. N'initialisez pas Firebase dans chaque écran.
2. Après avoir codé la connexion, fermez puis relancez l'application pour vérifier que la session est restaurée.
3. Si vous ajoutez une cible Web, prévoyez une initialisation et une persistance adaptées à cette plateforme. Le test obligatoire reste mobile.

Repère technique : [Firebase : API Authentication et persistance](#)

RÉSULTAT ATTENDU

Les services disposent d'une instance Firebase commune et la persistance mobile est préparée.

AuthContext

L'ensemble des écrans doit connaître la même session, y compris pendant son chargement initial.

01 État partagé à prévoir

Propriété	Signification
user	User Firebase ou null quand la personne est déconnectée.
initializing	La première réponse d'Authentication n'est pas encore arrivée.
profile / profileLoading	Le profil Firestore et son chargement, gérés séparément.
error	Un éventuel échec de lecture du profil à présenter clairement.

02 Point de départ du fournisseur

```
useEffect(() => {  
  const stop = onAuthStateChanged(auth, (account) => {  
    setUser(account);  
    setInitializing(false);  
  });  
  return stop;  
}, []);
```

À intégrer dans votre AuthProvider avec les imports de React, de firebase/auth et de votre fichier firebase.ts. Créez ensuite useAuth() pour accéder au contexte.

03 Travail demandé

1. Installez AuthProvider autour de la navigation racine. Pendant initializing, affichez un indicateur d'attente.
2. À chaque changement d'UID, effacez l'ancien profil et ses erreurs. Chargez ensuite users/{uid} pour le nouveau compte.
3. Un profil manquant ouvre le parcours de complétion. Une erreur réseau ouvre un état d'erreur avec Réessayer.
4. Arrêtez les écoutes au démontage et à la déconnexion. Protégez les routes connectées.

04 À expliquer

Pourquoi auth.currentUser lu une seule fois ne suffit-il pas pour piloter une interface dont la session peut changer ?

RÉSULTAT ATTENDU

La navigation suit l'état réel de la session, sans afficher brièvement les données d'un autre compte.

Inscription utilisateur

Créez une identité, puis accompagnez la personne jusqu'à la création de son profil sportif.

01 Formulaire à développer

Champ	Contrôle attendu
Nom affiché	2 à 40 caractères après nettoyage des espaces.
Email	Champ requis, format vérifié, clavier adapté.
Mot de passe	Conforme à la politique du projet, masqué à l'écran.
Confirmation	Strictement identique au mot de passe.

02 Repère d'API à intégrer dans votre service

```
const credential = await createUserWithEmailAndPassword(  
  auth, email.trim(), password  
);  
const uid = credential.user.uid;  
// À vous : créer users/{uid} avec les champs autorisés.  
// Puis diriger vers la complétion du profil sportif.
```

Importez `createUserWithEmailAndPassword` depuis `firebase/auth`. Pour créer le profil, utilisez une référence `doc(db, "users", uid)` et `setDoc`. La règle doit autoriser uniquement le propriétaire et les champs prévus.

03 Travail demandé

1. Validez le formulaire avant l'appel réseau. Affichez une attente et empêchez une deuxième soumission.
2. Créez le compte, puis le document de profil initial avec `gymId: null`, `createdAt` et `updatedAt` côté serveur.
3. Si le compte existe mais que l'écriture du profil échoue, proposez de reprendre cette étape. Ne relancez pas automatiquement une nouvelle inscription.
4. Après réussite, vérifiez l'identité dans Authentication et le document portant exactement le même UID dans Firestore.

04 Test de sortie

Une inscription valide fonctionne. Un email déjà utilisé ou une erreur de réseau ne bloque pas l'application. Après un échec de création du profil, le compte peut reprendre le parcours sans dupliquer son identité.

RÉSULTAT ATTENDU

Un compte est créé et l'utilisateur peut compléter son profil, même après un échec partiel.

Connexion utilisateur

Connectez l'utilisateur, chargez son profil et permettez une déconnexion qui nettoie toute l'interface.

01 Opérations du service à implémenter

```
await signInWithEmailAndPassword(auth, email.trim(), password);  
// AuthContext reçoit ensuite la session et charge le profil.
```

```
await signOut(auth);  
// AuthContext et les abonnements vident les données locales.
```

Importez ces fonctions depuis `firebase/auth`. Gérez les erreurs dans le service ou l'écran selon l'organisation retenue, avec un contrat clair.

02 Décisions après connexion

État détecté	Comportement de l'application
Session en cours de vérification	Chargement visible, navigation protégée en attente.
Identifiants invalides	Message générique et formulaire à nouveau utilisable.
Profil absent ou incomplet	Création / complétion du profil, puis choix de salle.
Profil complet avec gymId	Ouverture de Découvrir et chargement des membres.
Échec de lecture Firestore	Message réseau ou accès, avec possibilité de réessayer.

03 À coder

1. Placez une action Se déconnecter sur le profil. Attendez la réussite de `signOut` avant de confirmer la déconnexion.
2. Videz listes, messages, profil et erreurs du compte précédent. Arrêtez les écoutes associées à cet UID.
3. Vérifiez qu'un retour arrière ou un lien direct ne redonne pas accès aux écrans privés une fois déconnecté.
4. Fermez et relancez l'application pour tester la restauration de session prévue page 15.

04 Test à deux comptes

Connectez A, déconnectez-le puis connectez B sur le même appareil. Aucun nom, demande ou message de A ne doit apparaître pendant le chargement de B.

RÉSULTAT ATTENDU

La session est stable, restaurée au lancement et correctement nettoyée à la déconnexion.

Profil sportif

Le profil présenté aux autres membres contient uniquement les informations utiles pour organiser un entraînement.

01 Champs du document `users/{uid}`

Champs	Format et consigne
<code>displayName</code> · <code>city</code>	Nom affiché : 2-40 caractères ; ville : 2-60.
<code>level</code>	<code>beginner</code> , <code>intermediate</code> ou <code>advanced</code> ; libellés français à l'écran.
<code>goal</code> · <code>sports</code>	Un objectif requis ; tableau contenant au moins un sport.
<code>availability</code>	Liste : jour (1-7), début et fin en minutes depuis minuit.
<code>bio</code>	Texte facultatif de 280 caractères maximum.
<code>gymId</code>	null au départ ; identifiant d'une salle existante après sélection.
<code>createdAt</code> · <code>updatedAt</code>	Dates de création et de dernière modification.

02 Construire puis connecter le formulaire

1. Au jalon J2, gérez les valeurs avec `useState`. Utilisez des choix contrôlés pour le niveau, les sports et les jours.
2. Validez au moins une disponibilité : jour valide, $0 \leq \text{début} < \text{fin} \leq 1440$. Affichez les erreurs près des champs concernés.
3. Au jalon J5, préremplissez depuis `users/{uid}` puis enregistrez les champs modifiables avec `updateDoc`.
4. N'autorisez que le propriétaire à modifier ce profil. Les champs autorisés, types, tailles et valeurs sont aussi contrôlés par les règles.

03 Visibilité et complétude

Autorisez un profil brouillon sans salle. Pour choisir sa salle, il doit posséder un nom, une ville, un niveau, un objectif, un sport et une disponibilité. Vérifiez aussi cette complétude dans les règles.

Dans cette V1, la salle est choisie une fois et reste fixe. Son changement est un bonus, car il impose de décider du devenir des demandes et relations existantes. L'email n'est pas recopié dans le profil.

RÉSULTAT ATTENDU

Le profil est validé, modifiable par son propriétaire et conservé après redémarrage.

Choix de la salle de sport

La V1 utilise un petit catalogue fictif. Une recherche par ville remplace le calcul de proximité géographique.

01 Jeu de données à créer dans gyms

Document ID	name	city
gym_sqy	Campus Gym SQY	Guyancourt
gym_versailles	Studio Forme Versailles	Versailles
gym_paris	Campus Training Paris	Paris

02 Structure d'une salle

Chaque document contient `name`, `city`, `address` et `createdAt`. Utilisez une adresse fictive explicitement marquée comme telle. Une image locale peut illustrer les cartes ; aucun service de cartographie n'est requis.

03 Travail demandé

1. Lisez les salles depuis Firestore et affichez-les dans une `FlatList` avec une clé stable.
2. Ajoutez un champ de recherche par nom ou ville. Ce filtrage local suffit pour ce petit catalogue.
3. Au choix d'une salle, affichez une confirmation. Enregistrez son identifiant dans `users/{uid}.gymId`.
4. Les règles vérifient que la salle existe, que le profil appartient à l'utilisateur et que le `gymId` initial est encore nul.
5. Après enregistrement, chargez les membres de cette salle. Au retour sur le profil, la salle sélectionnée doit être visible.

04 Préparer les comptes de démonstration

Compte fictif	Affectation
A · Lina	gym_sqy ; niveau beginner ; objectif remise en forme.
B · Sami	gym_sqy ; niveau intermediate ; objectif régularité.
C · Nora	gym_paris ; niveau beginner ; objectif cardio.

RÉSULTAT ATTENDU

A et B sont rattachés à la même salle ; C sert à tester le filtrage et les droits.

Affichage des membres de la salle

Montrez les profils pertinents à partir du gymId de l'utilisateur connecté, puis permettez la consultation d'un membre.

01 Requête de départ

```
const membersQuery = query(
  collection(db, 'users'),
  where('gymId', '==', currentProfile.gymId)
);
// À vous : charger les documents, mapper id + data,
// puis retirer le profil dont id === user.uid.
```

Importez collection, query, where et getDocs depuis firebase/firestore. Lancez cette requête uniquement après le chargement d'un profil possédant un gymId valide.

02 À coder

1. Créez listGymMembers(gymId) et retournez des profils typés. Réutilisez MemberCard construite avec les données locales.
2. Affichez le nom, le niveau, l'objectif et les disponibilités. Un appui ouvre le détail avec userId.
3. Masquez votre propre carte. Ajoutez un état vide si la salle ne contient aucun autre membre.
4. Préparez une règle de lecture limitée au propriétaire ou aux membres de la même salle. La requête inclut donc le filtre gymId compatible avec cette règle.

03 Vérification avec A, B et C

Action	Résultat attendu
A ouvre Découvrir	B est visible ; A et C ne sont pas affichés.
A ouvre le profil B	Les informations sportives autorisées apparaissent.
A tente de lire directement le profil C	Accès refusé par les règles, sauf sa propre identité.
A se trouve seul dans une salle	Message vide utile et action de rechargement.

04 Indice

Les règles Firestore ne filtrent pas une requête à votre place. Charger tous les utilisateurs puis masquer C dans React ne réalise pas le contrôle d'accès demandé.

RÉSULTAT ATTENDU

Chaque utilisateur voit les autres membres autorisés de sa salle, sans fuite de profils.

Envoi d'une demande d'entraînement

Une demande propose une séance précise à un autre membre de la même salle.

01 Formulaire à développer

Champ	Validation avant envoi
Sport	Valeur choisie dans les sports proposés par l'application.
Date et heures	Début dans le futur ; fin strictement après le début.
Message	Facultatif, 500 caractères maximum après nettoyage.
Destinataire	Autre utilisateur existant, dans la même salle.

02 Du formulaire à l'écriture

1. Récupérez l'UID connecté depuis AuthContext. Relisez le profil destinataire et vérifiez sa salle.
2. Construisez les instants de début et de fin à partir de la date et des heures locales. Stockez-les en Timestamp, avec timeZone: "Europe/Paris" pour le jeu de test.
3. Préparez le document décrit page 23 : expéditeur, destinataire, participants, salle, sport, créneau et statut pending.
4. Générez une seule référence de document pour cette soumission et réutilisez son ID en cas de nouvelle tentative. Désactivez le bouton pendant l'envoi.
5. Après confirmation de l'écriture, revenez à Mes demandes envoyées et affichez la nouvelle demande.

03 Règles métier à appliquer

Interdictions : s'envoyer une demande, viser un membre d'une autre salle, proposer un créneau passé ou créer directement un statut accepted.

Une nouvelle tentative du même envoi doit conserver le même identifiant. Si le document existe déjà, relisez-le et informez l'utilisateur. La prévention de propositions distinctes mais similaires est un bonus.

04 Test de sortie

A propose à B une séance demain de 18 h à 19 h. Vérifiez le document enregistré, son statut et son affichage chez A. Testez aussi une date passée et un double appui.

RÉSULTAT ATTENDU

Une demande valide et unique pour cette soumission est enregistrée avec le statut pending.

Collection trainingRequests

Le document de demande constitue la preuve du créneau proposé et de la décision du destinataire.

01 Schéma minimal d'une demande

Champ	Type / rôle
senderId · receiverId	UID de l'expéditeur et du destinataire.
members	Tableau des deux UID, sans doublon.
gymId · sport	Salle commune et activité proposée.
startsAt · endsAt	Timestamp ; début futur lors de la création.
timeZone · message	Zone d'affichage du créneau ; texte facultatif.
status	pending, accepted ou rejected.
createdAt · respondedAt	Horodatage serveur ; respondedAt est nul avant la réponse.

02 Requêtes à préparer pour les deux listes

```
// Liste des demandes reçues
where('receiverId', '==', user.uid)
// Liste des demandes envoyées
where('senderId', '==', user.uid)
```

Ajoutez ces conditions à une requête sur trainingRequests. Un filtre status et un tri par createdAt peuvent nécessiter un index composite ; créez celui demandé et consignez-le dans firestore.indexes.json.

03 Travail demandé

1. Créez les vues Reçues et Envoyées, éventuellement dans un même écran à deux onglets.
2. Affichez l'autre sportif, le sport, la date, les heures et un badge En attente, Acceptée ou Refusée.
3. La création fixe les participants et le créneau. Lors d'une réponse, seuls le statut et la date de réponse changent.
4. Vérifiez que A et B peuvent lire leur demande et que C ne peut pas y accéder, même en connaissant son ID.

RÉSULTAT ATTENDU

Les listes reçues et envoyées présentent le même document et un statut cohérent.

Acceptation / refus

Seul le destinataire répond à une demande encore en attente. L'acceptation doit produire un ensemble de données cohérent.

01 Transitions autorisées

Avant	Action du destinataire	Après
pending	Refuser	rejected + respondedAt.
pending	Accepter un créneau encore futur	accepted + relation + chat + séance.
accepted / rejected	Répondre une deuxième fois	Aucune nouvelle transition.

02 Acceptation avec runTransaction

1. Calculez les références : demande, relation de la paire, chat de cette relation et séance dont l'ID est celui de la demande.
2. Lisez la demande puis la relation, avant toute écriture. Vérifiez le destinataire connecté, le statut pending et un début de séance encore futur.
3. Si la relation est absente, créez-la avec son chat ; sinon, réutilisez-les. Marquez la demande accepted et créez la séance depuis les données originales.
4. Validez le tout dans la même transaction. Les règles contrôlent aussi cette cohérence, avec `getAfter()` pour les documents écrits ensemble.
5. Affichez le succès uniquement après la validation. En cas de réseau coupé, proposez Réessayer : une transaction nécessite une connexion.

03 Refus et nouvelle tentative

Un refus met seulement à jour la demande. Il ne crée ni relation, ni séance, ni chat. Si A et B avaient déjà une relation issue d'une autre demande, celle-ci reste inchangée.

Si une acceptation est relancée après réussite, lisez les documents existants et retournez un résultat déjà traité. Les identifiants stables empêchent de créer une deuxième séance.

04 Point de contrôle

Deux validations concurrentes produisent une seule séance. Les règles doivent autoriser la lecture de la relation encore absente pour cette paire, selon la page 25.

Repère technique : [Firestore : transactions et getAfter](#)

RÉSULTAT ATTENDU

La décision du destinataire est appliquée une seule fois et l'acceptation reste cohérente.

Création d'une relation

Une relation représente un lien accepté entre deux sportifs. Plusieurs séances peuvent ensuite utiliser ce même lien.

01 Un identifiant stable pour la paire

```
// Pour les UID générés par Authentication dans ce TP :  
const members = [senderId, receiverId].sort();  
const pairId = members.join('__');  
// relations/{pairId} et chats/{pairId}
```

Le calcul doit donner le même résultat pour A/B et B/A. Centralisez-le dans une fonction. Des identifiants personnalisés demanderaient un encodage non ambigu et sa validation dans les règles.

02 Document relations/{pairId}

Champ	Contenu attendu
members	Deux UID distincts, dans l'ordre canonique choisi.
gymId	Salle de la première demande acceptée.
sourceRequestId	ID de la demande qui a créé la relation.
createdAt	Date serveur de création ; conservée lors des séances suivantes.

03 Travail demandé

1. Dans la transaction de la page 24, créez la relation uniquement si elle n'existe pas encore.
2. Les règles vérifient les participants, la salle, l'ID de paire et la demande source acceptée. L'appel doit provenir du destinataire de cette demande.
3. Développez Mes relations avec une requête `where("members", "array-contains", user.uid)`.
4. Affichez le partenaire et un bouton Ouvrir le chat. Résolvez le nom depuis son profil autorisé ; ne fondez aucun droit sur le nom affiché.

04 Scénario de contrôle

A et B acceptent deux demandes différentes : **une relation, un chat et deux séances**. La deuxième acceptation conserve la relation et les messages.

Lecture avant création : distinguez `get` de `list` dans les règles. Pour une relation absente, autorisez `get` si la clé de paire valide contient l'UID connecté. Pour un document existant, vérifiez ses membres. Ne lisez pas `resource.data` sans traiter le cas absent.

RÉSULTAT ATTENDU

La relation existe réellement dans Firestore et reste unique pour les deux utilisateurs.

Création d'une séance sportive

Une demande acceptée devient un entraînement visible dans le planning des deux participants.

01 Document trainingSessions/{requestId}

Champ	Règle de construction
requestId · relationId	Demande acceptée et paire à laquelle appartient la séance.
members · gymId · sport	Valeurs recopiées depuis la demande validée.
startsAt · endsAt · timeZone	Créneau accepté, conservé sans modification.
status	planned à la création ; done ou cancelled ensuite.
createdAt · updatedAt	Horodatages serveur.

02 À coder

1. Dans la transaction d'acceptation, utilisez l'ID de la demande comme ID de séance. Ne générez pas de nouvel ID lors d'une nouvelle tentative.
2. Créez Mes séances en filtrant sur members avec array-contains. Séparez À venir et Historique.
3. Affichez le partenaire, la salle, le sport, la date et les heures. Formatez les Timestamp pour l'utilisateur, sans stocker uniquement un libellé de date.
4. Permettez à un participant d'annuler une séance planned. Il peut la marquer done seulement lorsque l'heure de fin est passée.
5. Les règles n'autorisent que ces changements de statut et updatedAt. Les participants et le créneau restent immuables.

03 Comportements attendus

Une séance annulée reste dans l'historique. Son annulation ne supprime pas la relation et ne ferme pas automatiquement le chat.

Une demande dont le début est déjà passé reste consultable mais ne peut plus être acceptée. Proposez un message invitant à envoyer une nouvelle demande avec un autre créneau.

04 Test de sortie

La même séance apparaît chez A et B. Si A l'annule, B retrouve le statut après actualisation. Pour le jeu de test, calculez un créneau futur au moment de la démonstration.

RÉSULTAT ATTENDU

Le planning reflète les demandes acceptées et chaque séance conserve un historique clair.

Chat après acceptation

Le chat s'appuie sur une relation enregistrée et validée. La présence d'un bouton dans l'interface ne suffit pas à accorder l'accès.

01 Document chats/{pairId}

Champ	Rôle
relationId	Même pairId que le document de relation.
members	Les deux UID de la relation.
createdAt	Date serveur de création.
lastMessage · updatedAt	Facultatifs : à ajouter ensuite si vous gérez leur cohérence.

02 Créer et ouvrir la conversation

1. Créez le chat dans la transaction de la première acceptation, uniquement s'il n'existe pas déjà.
2. Affichez Mes conversations avec une requête sur les chats contenant l'UID connecté dans members.
3. L'écran de conversation reçoit chatId. Il lit le chat autorisé puis vérifie sa relation et son appartenance avant d'abonner la liste des messages.
4. Prévoyez les cas route inconnue, conversation non accessible et échec réseau. N'affichez pas un historique laissé par le compte précédent.

03 Décisions d'accès à démontrer

Situation	Comportement attendu
A/B : aucune demande acceptée	Chat non disponible ; écriture refusée par les règles.
Première demande refusée	Aucune relation ni conversation créée.
Première demande acceptée	Chat accessible à A et B seulement.
C connaît le chatId	Lecture et écriture refusées.
Nouvelle demande A/B refusée	Le chat déjà créé reste accessible.

RÉSULTAT ATTENDU

Seuls les deux utilisateurs d'une relation issue d'une acceptation peuvent accéder au chat.

Messages en temps réel

Chaque message est un document de la sous-collection chats/{pairId}/messages. Une écoute actualise l'écran des deux utilisateurs.

01 Schéma du message à enregistrer

```
{
  senderId: user.uid,
  text: draft.trim(),
  createdAt: serverTimestamp(),
}
```

Validez un texte non vide de 1 à 1 000 caractères. Créez une référence stable pour l'envoi en cours et réutilisez-la si une tentative doit être reprise. Ne videz la saisie qu'après confirmation.

02 Écoute à intégrer et à compléter

```
const uid = user?.uid;
useEffect(() => {
  setMessages([]);
  setError('');
  if (!chatId || !uid) return;
  const q = query(
    collection(db, 'chats', chatId, 'messages'),
    orderBy('createdAt', 'desc'), limit(50)
  );
  return onSnapshot(q, (snap) => {
    const rows = snap.docs.map(d => ({ ...d.data(), id: d.id }));
    setMessages(rows.reverse());
  }, () => setError('Messages indisponibles. Réessayez.'));
}, [chatId, uid]);
```

Déclarez les états et imports nécessaires. Videz la liste lors d'un changement de chat. La requête prend les 50 derniers messages ; l'affichage les remet dans l'ordre chronologique.

03 Travail demandé

1. Distinguez les messages envoyés et reçus, avec le texte et une heure lisible. Prévoyez un horodatage encore en attente de confirmation serveur.
2. Vérifiez l'envoi A → B puis B → A sans rechargement manuel. Un message vide doit être refusé.
3. Quittez puis rouvrez le chat : une seule écoute doit rester active. En cas d'échec d'envoi, conservez le brouillon et affichez son état.

Repère technique : [Firestore : écoutes en temps réel](#)

RÉSULTAT ATTENDU

A et B échangent en direct, sans doublons d'affichage et avec un traitement des erreurs.

Sécurité et affichage conditionnel

Implémentez ces contrats dans `firestore.rules` au fil des jalons. Les contrôles doivent tenir même si un utilisateur modifie son application.

01 Contrats de règles à compléter et à tester

Données	Autorisations et validations obligatoires
gyms	Lecture connectée ; aucune écriture depuis le client.
users	Lecture du propriétaire ou de la même salle non nulle. Écriture du propriétaire : champs, types et tailles p. 19 ; brouillon sans salle, profil complet pour la choisir, <code>gymId</code> ensuite immuable.
trainingRequests	Lecture des deux participants. Création par <code>senderId</code> connecté : autre membre de la même salle, créneau futur, statut pending. Réponse du destinataire : pending vers accepted / rejected ; champs métier immuables.
relations + chats	Création cohérente par le destinataire d'une demande acceptée ; participants et salle identiques. IDs validés. Lecture des membres seulement. Aucune modification des membres ni suppression dans le socle.
trainingSessions	Création liée à une demande acceptée, ID = <code>requestId</code> . Lecture des membres ; seules les transitions planned vers cancelled ou done après la fin sont autorisées.
messages	Membre du chat et relation valide ; <code>senderId</code> = UID connecté, texte de 1-1 000 caractères et <code>createdAt</code> serveur. Messages immuables ; aucune lecture extérieure.

02 Contrôles transversaux

Vérifiez `request.auth`, les champs autorisés, leurs types, les identifiants et les horodatages. Comparez l'ancien et le nouveau document pour empêcher un changement de propriétaire. Pour l'acceptation, validez les documents associés avec `getAfter()`.

Contrôlez aussi les requêtes : filtre `gymId`, `senderId`, `receiverId` ou `members` selon l'écran. Utilisez le simulateur de règles ou les émulateurs Firebase pour les accès interdits.

Repère technique : [Firestore : conditions des règles](#)

RÉSULTAT ATTENDU

Un tiers ne peut ni lire un échange, ni forger une acceptation, ni écrire au nom d'un autre.

Tests fonctionnels

Utilisez A et B dans gym_sqy, puis C dans gym_paris. Notez chaque résultat réel dans docs/tests.md, avec la preuve ou l'anomalie.

01 Recette minimale à exécuter

Test	Résultat attendu
T01 · Compte + redémarrage	Profil conservé et session restaurée.
T02 · Passage de A à B	Aucune donnée de A affichée chez B.
T03 · Membres de la salle	A voit B ; C reste inaccessible.
T04 · Demande valide + double appui	Une soumission, une demande reçue.
T05 · Demandes interdites	Refus : soi-même, autre salle, date passée.
T06 · Refus de la première demande	Statut rejected ; aucun nouveau chat.
T07 · Acceptation + nouvelle tentative	Une relation, un chat, une seule séance.
T08 · Deuxième séance A/B	Même relation et chat ; nouvelle séance.
T09 · Messages A/B	Réception directe et historique retrouvé.
T10 · Accès direct de C	Lecture et envoi dans le chat refusés.
T11 · Écriture forgée	Refus : faux senderId ou acceptation par A.
T12 · Réseau coupé à l'acceptation	Erreur visible ; reprise sans doublons.

02 Preuves attendues

Pour chaque test : étapes, résultat attendu, résultat observé, OK/KO et correction si nécessaire. Les tests d'autorisation doivent inclure un appel refusé par Firebase, au-delà de l'interface. Complétez par la lecture du code critique : nettoyage des écoutes, immutabilité des participants et identifiants stables des opérations rejouées.

RÉSULTAT ATTENDU

Les scénarios principaux et les accès interdits sont vérifiés avec des résultats traçables.

Barème étudiant et livrables

Le rendu doit permettre de comprendre, installer et vérifier votre application sans dépendre de votre poste de développement.

01 Grille d'évaluation proposée / 20

Critère	Points
Interface React Native, navigation et états d'écran.	3
Authentification, profil sportif et persistance.	3
Choix de salle, membres filtrés et détail.	2
Demandes, décision, relation et séance cohérentes.	5
Chat conditionnel et messages en temps réel.	3
Règles d'accès et tests négatifs démontrés.	2
Structure du code, README, tests et explication.	2

02 Livrables obligatoires

1. Le dépôt du code source avec `package.json`, le fichier de verrouillage, `.env.example`, `firebase.rules` et les index nécessaires. Excluez `node_modules` et les clés d'administration.
2. Un `README.md` : objectif, fonctionnalités réalisées, versions, commandes d'installation et lancement, configuration Firebase et création des comptes de test.
3. Le dossier `docs/` : modèle de données, suivi des jalons, tableau de tests, captures des écrans et de Firestore, limites connues.
4. Une application exécutable sur appareil ou émulateur et une démonstration avec deux comptes : demande, acceptation, relation, séance puis chat.

03 Préparer votre explication

Montrez une portion de code que vous maîtrisez : composant réutilisable, service, transaction ou règle. Expliquez un bug rencontré et sa correction. Si le travail est réalisé en groupe, identifiez les contributions dans le suivi.

RÉSULTAT ATTENDU

Le rendu contient le code, les preuves de fonctionnement et une procédure de reprise claire.

Bonus et améliorations

Choisissez un bonus après validation du parcours obligatoire. Conservez un socle stable et expliquez le besoin auquel répond l'amélioration.

01 Pistes pour prolonger le projet

Niveau	Extension et difficulté à traiter
Accessible	Filtres par niveau / sport, recherche de partenaires et badges de demandes. Prévoir le cas sans résultat.
Intermédiaire	Photo de profil : sélection, compression, stockage et droits associés. Vérifier les conditions du service avant activation.
Intermédiaire	Historique et pagination des messages au-delà des 50 derniers. Préserver un ordre stable.
Avancé	Changement de salle : définir le devenir des demandes, relations, profils lisibles et séances existantes.
Avancé	Blocage d'un membre : stopper les nouvelles demandes et les messages dans les règles, puis adapter l'interface.
Avancé	Carte, distance ou notifications : permissions, environnement mobile compatible, refus de permission et reprise.

02 Défi de conception

Comment éviter deux demandes concurrentes pour le même partenaire et le même créneau ? Proposez une clé d'unicité, une transaction et les règles correspondantes. Expliquez comment traiter une proposition dans le sens inverse.

03 Votre projet est terminé lorsque...

Un autre étudiant peut installer l'application, réaliser le parcours complet à deux comptes et comprendre vos choix. Vous pouvez montrer les données créées et expliquer pourquoi un troisième utilisateur ne peut pas accéder au chat.

RÉSULTAT ATTENDU

Vous disposez d'une application mobile démontrable et d'une base solide pour aller plus loin.